COMM CAST

PROBLEM STATEMENT

A facilities management company operates multiple commercial buildings and monitors their electricity consumption at regular intervals. Energy usage changes with occupancy, building size, operating hours, outdoor temperature, humidity, HVAC operating mode, and day type. The company wants a machine-learning solution that learns from historical building readings and predicts electricity consumption for upcoming intervals.

The exported data may contain duplicate reading IDs, missing numeric values, inconsistent category formatting, and invalid timestamps. Build a reusable Python pipeline that cleans the data, trains and evaluates two regression models, selects the better model, and generates predictions for scheduled future readings.

OBJECTIVE

Implement a clear four-stage pipeline: prepare the data, train two regression models, compare their held-back performance, and predict electricity consumption for new records. The same fitted preprocessing steps must be used for training, evaluation, and prediction.

FILE STRUCTURE

```
Project
■■■ data
■   ■■■ building_train.csv
■   ■■■ building_test.csv
■   ■■■ building_predict.csv
■■■ processed_data
■■■ artifacts
■■■ output
■■■ main.py
■■■ requirements.txt
```

DATASET DESCRIPTION

Column	Role	Description
reading_id	Record	Unique building-reading reference.
reading_ts	Record	Date and time when the energy reading was recorded.
building_code	Feature	Commercial building identifier.
floor_area_sqm	Feature	Total floor area of the building in square metres.
occupancy_count	Feature	Number of people present during the interval.
operating_hours	Feature	Number of hours the building operated during the interval.
outdoor_temp_c	Feature	Outdoor temperature in degrees Celsius.
humidity_pct	Feature	Relative humidity percentage.
hvac_mode	Feature	cooling, heating, ventilation_only, or off.
day_type	Feature	weekday, weekend, or public_holiday.
energy_consumed_kwh	Target	Metered electricity consumption in kilowatt-hours.

building_test.csv has the same columns as building_train.csv. building_predict.csv contains all record and feature columns but does not contain energy_consumed_kwh.

SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
reading_id	ER-2026-018420	ER-2026-026315	ER-2026-031908
reading_ts	2026-04-18 09:00	18/03/2026 18:30	2026-05-26 14:00
building_code	BLR-A1	HYD-B2	BLR-A1
floor_area_sqm	8200	12500	6400

Column	Record 1	Record 2	Record 3
occupancy_count	340	510	125
operating_hours	9	12	(blank)
outdoor_temp_c	32.1	24.8	29.4
humidity_pct	61	72	55
hvac_mode	Cooling	heating	VENTILATION_ONLY
day_type	Weekday	weekend	Public_Holiday
energy_consumed_kwh	1865.4	2410.8	735.2

TASK

Implement the following functions and the main block in main.py. Function names and return values are mandatory.

FUNCTION 1: CLEAN AND PREPARE DATA

prepare_data(input_path, output_path, training=False)

- Read the CSV file using pandas and remove duplicate reading_id values, keeping the first record.
- Parse reading_ts using errors="coerce" and remove rows containing an invalid timestamp.
- Trim spaces and convert building_code, hvac_mode, and day_type to lowercase.
- Convert numeric columns using pandas.to_numeric(..., errors="coerce"). Fill missing numeric feature values with the median calculated from the current dataset.
- Create two features: reading_hour from reading_ts and occupancy_density = occupancy_count / floor_area_sqm. Avoid division by zero.
- One-hot encode building_code, hvac_mode, and day_type using pandas.get_dummies().
- Write the prepared dataset to output_path without the DataFrame index and return the DataFrame.

FUNCTION 2: TRAIN MODELS

train_models(processed_train_path, preprocessor_path, linear_model_path, forest_model_path)

- Separate reading_id and energy_consumed_kwh from the predictor columns.
- Create and fit a preprocessing pipeline containing median imputation and StandardScaler for numeric predictor columns.
- Train LinearRegression as the baseline model.
- Train RandomForestRegressor with n_estimators=150, max_depth=10, min_samples_leaf=2, random_state=42, and n_jobs=-1.
- Save the fitted preprocessor and both models using joblib. Return both fitted models.

FUNCTION 3: EVALUATE AND SELECT

evaluate_and_select(test_path, preprocessor_path, linear_model_path, forest_model_path, report_path)

- Load the prepared test dataset, preprocessor, and both saved models. Do not refit any object.
- For each model, calculate MAE, RMSE, and R^2 for energy_consumed_kwh.
- Create a comparison DataFrame with columns model_name, mae, rmse, and r2, ordered by rmse ascending.
- Select the model having the lowest RMSE. Save the comparison table to report_path and return the selected model path and comparison DataFrame.

FUNCTION 4: PREDICT NEW DATA

predict_energy(model_path, preprocessor_path, input_path, processed_path, output_path)

- Prepare the scheduled dataset using the same cleaning and feature logic.
- Align the scheduled predictor columns to the training feature order. Add missing dummy columns with 0 and ignore unexpected dummy columns.
- Load the selected model and fitted preprocessor, transform the scheduled data, and predict energy consumption.
- Replace negative predictions with 0 and round values to one decimal place.
- Write reading_id and predicted_energy_consumed_kwh to output_path without an index. Preserve the input order and return the prediction DataFrame.

MAIN BLOCK

- Create processed_data, artifacts, and output directories when they do not exist.
- Prepare the training and test datasets.
- Train both models, compare the models, and select the model with the lowest held-back RMSE.

- Prepare the scheduled dataset and generate predictions using the selected model.
- Print prepared dataset shapes, model comparison, selected model path, and the first five predictions. Do not print inside the functions.

GENERATED FILES

Path	Contents
processed_data/building_train_cleaned.csv	Prepared training rows and target.
processed_data/building_test_cleaned.csv	Prepared held-back rows and target.
processed_data/building_predict_cleaned.csv	Prepared scheduled rows aligned to training features.
artifacts/preprocessor.pkl	Fitted preprocessing pipeline.
artifacts/linear_regression.pkl	Baseline model.
artifacts/random_forest.pkl	Random forest model.
output/model_comparison.csv	MAE, RMSE, and R^2 for both models.
output/building_predictions.csv	Electricity-consumption predictions for scheduled readings.

EXPECTED OUTPUT

```
Prepared training rows: (4200, 18)
Prepared test rows: (1000, 18)

Model comparison:
       model_name          mae      rmse     r2
       random_forest      82.4     118.7    0.92
       linear_regression  131.6    184.2    0.81

Selected model: artifacts/random_forest.pkl
Predictions written: (300, 2)
```

The values above are illustrative. Exact metrics may vary, but function signatures, output filenames, validation requirements, and output columns must match the specification.